

Day 13 — Linux Socket Programming & TCP Client/Server

Today we move from **networking concepts** → **actual C/C++ programming**.

This is especially important for your interview because you already know the theory of:

Application

↓

TCP

↓

IP

↓

Ethernet

↓

NIC

Today you'll understand how Linux socket APIs create that communication.

Day 13 Goals

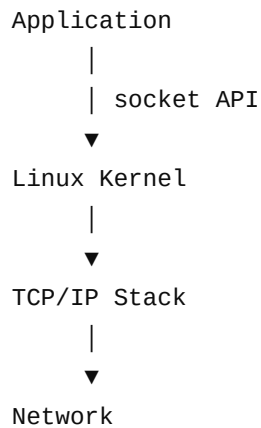
By the end of today, you should understand:

1. What is a socket?
 2. Socket vs port vs IP address
 3. TCP client/server architecture
 4. `socket()`
 5. `bind()`
 6. `listen()`
 7. `accept()`
 8. `connect()`
 9. `send()` / `recv()`
 10. `close()`
 11. How each API maps to the TCP stack
 12. TCP server lifecycle
 13. TCP client lifecycle
 14. What happens inside the Linux kernel
 15. Hands-on C program
 16. Important interview questions
-

1. What is a Socket?

A **socket** is an operating-system interface that an application uses to communicate over a network.

Think of it as:



Your application does not directly manipulate Ethernet frames or TCP headers.

Instead, it uses APIs such as:

```
socket()  
bind()  
listen()  
accept()  
connect()  
send()  
recv()  
close()
```

2. Socket vs IP vs Port

These three are often confused.

Suppose:

```
Server IP    = 192.168.1.100  
Server Port  = 5000  
Protocol     = TCP
```

Conceptually:

192.168.1.100:5000



Socket

But a socket is more than just the text `IP:port` ; it is an OS-managed communication endpoint.

3. Real-World Analogy

Think about a company office.

Building



IP address

Department



Port

Communication endpoint



Socket

So:

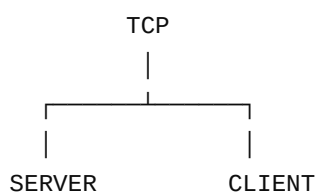
IP → Which machine?

Port → Which service?

Socket → Application's communication endpoint

4. TCP Client/Server

The basic architecture is:



Server:

```
socket()
  ↓
bind()
  ↓
listen()
  ↓
accept()
  ↓
recv()/send()
```

Client:

```
socket()
  ↓
connect()
  ↓
send()/recv()
```

This sequence is **very important**.

5. Complete TCP Server Flow

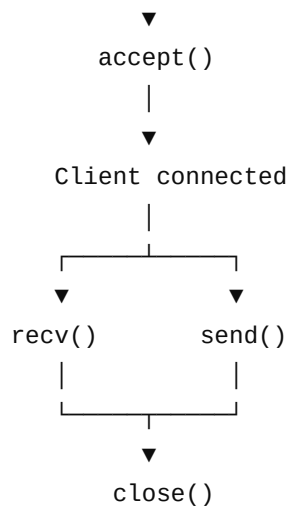
Memorize:

```
socket()
  ↓
bind()
  ↓
listen()
  ↓
accept()
  ↓
recv()/send()
  ↓
close()
```

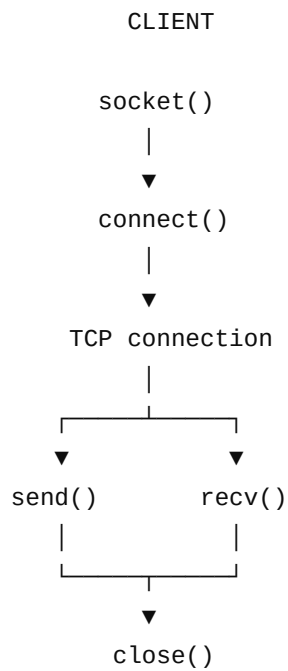
Visualize:

SERVER

```
socket()
  |
  ▼
bind()
  |
  ▼
listen()
  |
```



6. Complete TCP Client Flow



7. socket()

The first API is:

```
int socket(int domain, int type, int protocol);
```

Example:

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

Parameters

domain

Specifies the address family.

Common:

AF_INET

for IPv4.

AF_INET6

for IPv6.

So:

```
socket(AF_INET, ...)
```

means:

Create an IPv4 socket.

type

Specifies the communication type.

For TCP:

SOCK_STREAM

For UDP:

SOCK_DGRAM

So:

```
socket(AF_INET, SOCK_STREAM, 0);
```

means:

```
IPv4  
+  
TCP-style byte stream
```

protocol

Usually:

```
0
```

means:

Let the system select the appropriate protocol for the specified domain/type.

8. What Does `socket()` Actually Do?

This is important for your previous question.

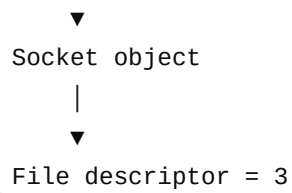
When you call:

```
socket(AF_INET, SOCK_STREAM, 0);
```

Linux creates an endpoint represented to your process by a **file descriptor**.

Example:

```
Application  
|  
| socket()  
▼  
Linux Kernel  
|
```



For example:

```
int fd = socket(...);
```

might return:

```
fd = 3
```

The number can differ.

9. Is `socket()` a TCP Connection?

No.

This is an important interview point.

```
socket()
```

creates a socket endpoint.

It does **not** establish a TCP connection to another machine.

Think:

```
socket()
  ↓
Create communication endpoint
```

not:

```
socket()
  ↓
TCP connection
```

The TCP connection is established later by:

```
connect()
```


on the client side, or through an incoming connection handled by:

```
accept()
```

on the server side.

10. bind()

Server normally uses:

```
bind()
```

Prototype:

```
int bind(  
    int sockfd,  
    const struct sockaddr *addr,  
    socklen_t addrlen  
);
```

Example:

```
bind(sockfd,  
    (struct sockaddr *)&server_addr,  
    sizeof(server_addr));
```

11. What Does bind() Mean?

bind() associates the socket with a **local address**.

For example:

```
192.168.1.100:5000
```

Conceptually:

```
Socket
|
▼
Local IP + Port
|
▼
192.168.1.100:5000
```

It tells the kernel:

"This socket should use this local address."

12. Important Correction From Your Earlier Question

You previously asked:

Does `bind()` connect to the Transport Layer again?

No.

`bind()` does not establish a remote TCP connection.

It configures the local endpoint.

```
bind()
↓
Local address/port association
```

Whereas:

```
connect()
↓
Request TCP connection to remote endpoint
```

13. Example `bind()`

```
struct sockaddr_in server_addr;

server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(5000);
server_addr.sin_addr.s_addr = INADDR_ANY;

bind(sockfd,
```

```
(struct sockaddr *)&server_addr,
```



Now let's understand each field.

14. struct sockaddr_in

Used for IPv4 socket addresses.

Important members:

```
struct sockaddr_in {  
    sa_family_t    sin_family;  
    in_port_t      sin_port;  
    struct in_addr sin_addr;  
};
```

`sin_family`

```
sin_family = AF_INET;
```

Means:

IPv4 address.

`sin_port`

```
sin_port = htons(5000);
```

Means:

Port 5000.

Why `htons()` ?

Because network protocols use **network byte order**.

We'll cover byte ordering later today.

`sin_addr`

Example:

```
sin_addr.s_addr = INADDR_ANY;
```

Means the server can listen on the local machine's available IPv4 interfaces.

Conceptually:

```
192.168.1.100:5000  
10.0.0.5:5000  
127.0.0.1:5000
```

depending on the interfaces/configuration.

15. INADDR_ANY

This is commonly used on servers:

```
server_addr.sin_addr.s_addr = INADDR_ANY;
```

Meaning approximately:

Bind to all local IPv4 interfaces.

Compare:

```
INADDR_ANY  
↓  
All local IPv4 interfaces
```

versus:

```
192.168.1.100  
↓  
Specific local interface/address
```

16. listen()

After `bind()` :

```
listen(sockfd, backlog);
```

Example:

```
listen(sockfd, 10);
```

`listen()` changes the socket into a **passive listening socket** for incoming TCP connections.

17. What Does `listen()` Do?

Think:

```
socket()
  ↓
Create endpoint
  ↓
bind()
  ↓
Give it local address/port
  ↓
listen()
  ↓
Ready for incoming connections
```

It does **not** mean a client has connected yet.

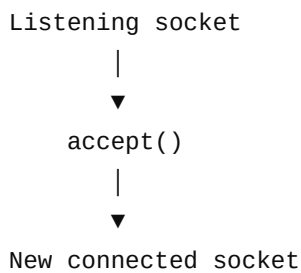
18. `accept()`

The server then calls:

```
int client_fd = accept(sockfd, ...);
```

This waits for an incoming connection.

Conceptually:



This is one of the most important concepts in socket programming.

19. Why Does `accept()` Return Another FD?

Suppose:

```
Listening socket = 3
```

A client connects.

Then:

```
accept()
```

might return:

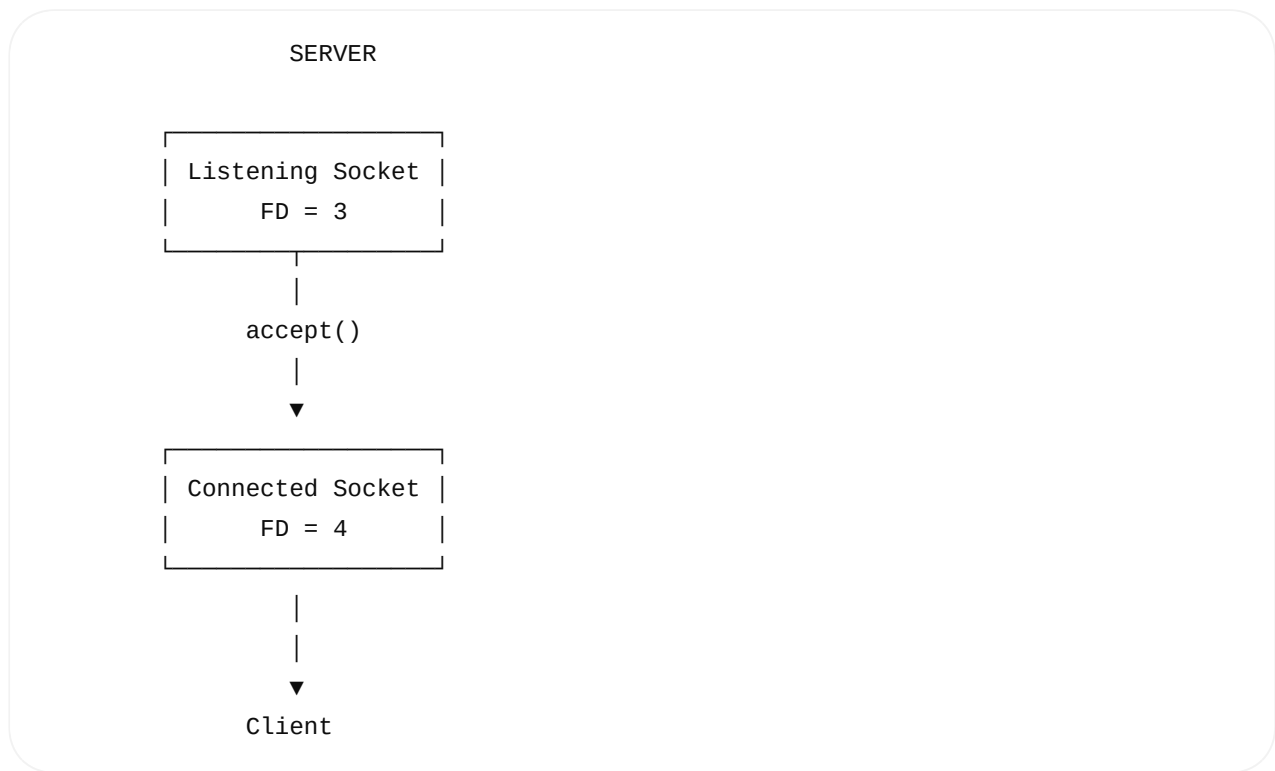
```
client socket = 4
```

Now:

```
FD 3
  ↓
Listening socket

FD 4
  ↓
Connected client socket
```

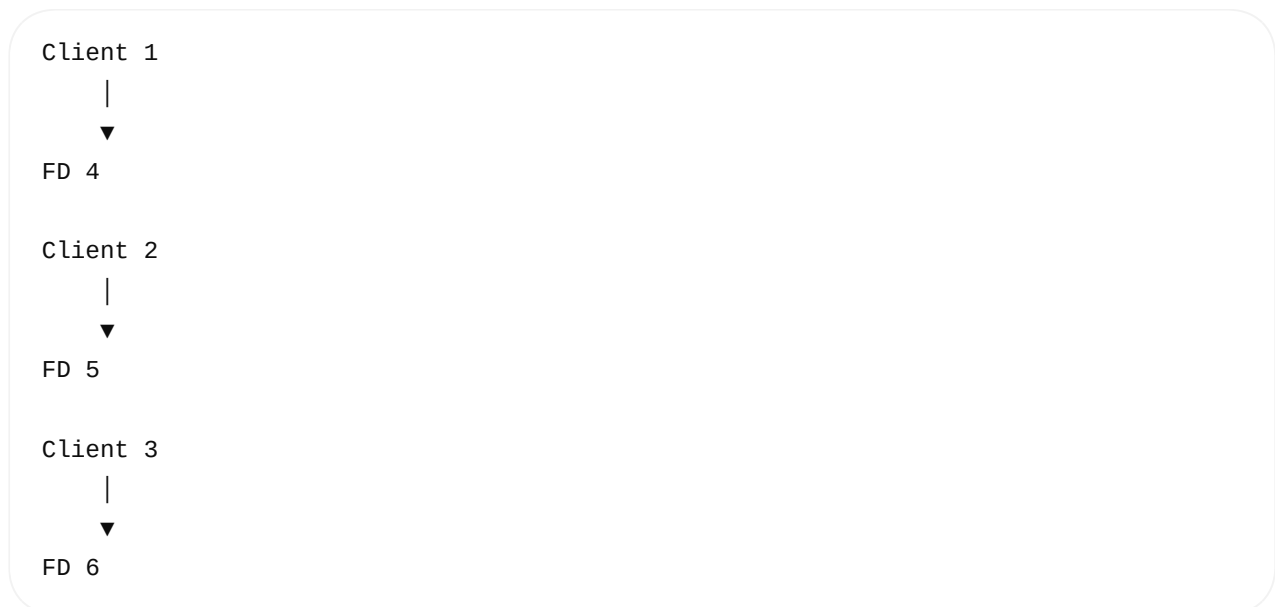
20. Visualize Server Sockets



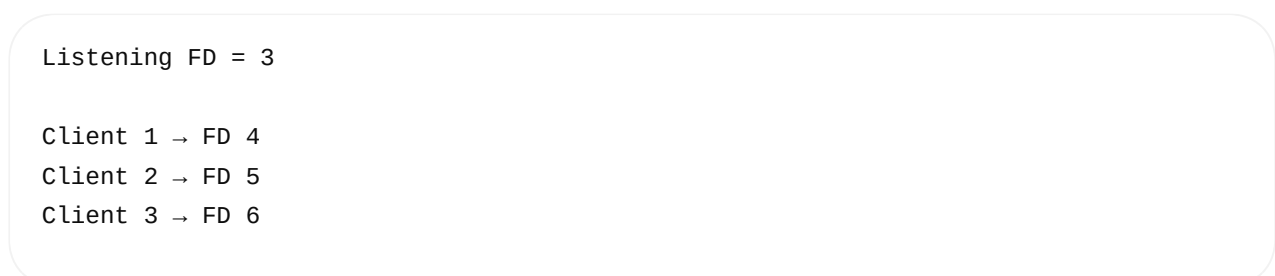
The listening socket remains available to accept additional connections.

21. Why Is This Design Useful?

Suppose three clients connect:



Server:



This is how a server can manage multiple connections.

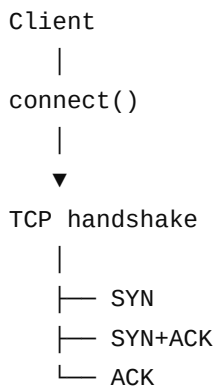
22. connect()

Client normally calls:

```
connect(sockfd,  
        (struct sockaddr *)&server_addr,  
        sizeof(server_addr));
```

This requests a connection to the remote endpoint.

For TCP:



So your previous learning now connects directly to the API.

23. Does connect() Directly Call TCP?

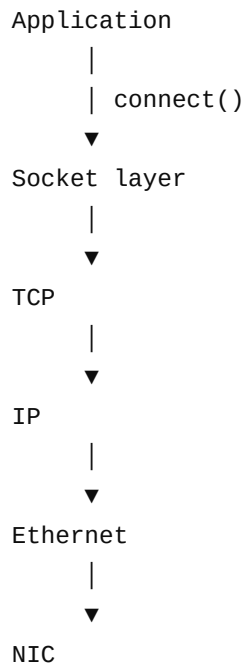
At the application level:

```
connect()
```

is a socket API.

The kernel processes the request and, for a TCP socket, causes the TCP stack to initiate the connection.

Conceptually:



So don't say:

" connect () directly calls Layer 4."

Better:

connect () is a socket API provided by the kernel; for a TCP socket, the kernel's networking stack uses TCP to establish the connection.

24. send ()

After connection:

```
send(sockfd, buffer, length, 0);
```

Example:

```
char msg[] = "Hello";  
  
send(sockfd, msg, strlen(msg), 0);
```

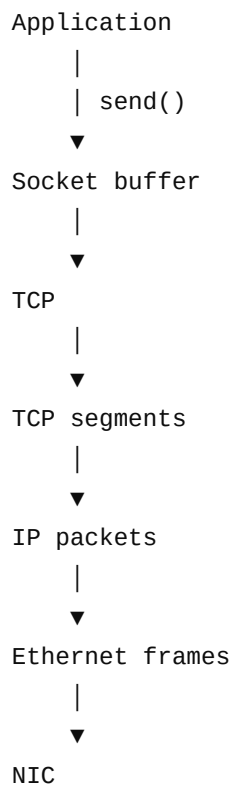
Parameters:

sockfd
buffer



25. What Does `send()` Do?

Conceptually:



This is exactly the flow you were asking about earlier.

26. Important: `send()` Does Not Mean One Segment

Suppose:

```
send(fd, buffer, 100000, 0);
```

It does **not** mean:

```
One TCP segment = 100000 bytes
```

TCP is a byte-stream protocol.

The kernel may split the data into multiple segments according to things such as:

MSS
TCP congestion control
send buffer
network conditions

So:

send()
↓
TCP byte stream
↓
Multiple segments as needed

27. recv()

Receiver uses:

```
recv(sockfd, buffer, sizeof(buffer), 0);
```

Conceptually:

NIC
↓
Ethernet
↓
IP
↓
TCP
↓
TCP receive buffer
↓
recv()
↓
Application

28. Important: TCP Is a Byte Stream

Suppose client sends:

```
send("Hello")  
send("World")
```

Server may receive:

HelloWorld

or:

Hello

then:

World

or another segmentation.

TCP does **not preserve application message boundaries**.

This is a major interview question.

29. TCP vs UDP

TCP

Byte stream
Reliable
Ordered
Connection-oriented

UDP

Datagrams
No connection establishment
No built-in reliability
Message boundaries preserved

Very important:

TCP:
send() → recv()

doesn't guarantee one send() corresponds to one recv() .

30. close()

When finished:

```
close(sockfd);
```

The file descriptor is closed.

For TCP, closing a connected socket can initiate TCP connection termination depending on the state and whether data remains to be sent.

Conceptually:

```
Application
|
close()
▼
TCP connection termination
```

Often you'll observe:

```
FIN
ACK
...
```

31. Complete TCP Server

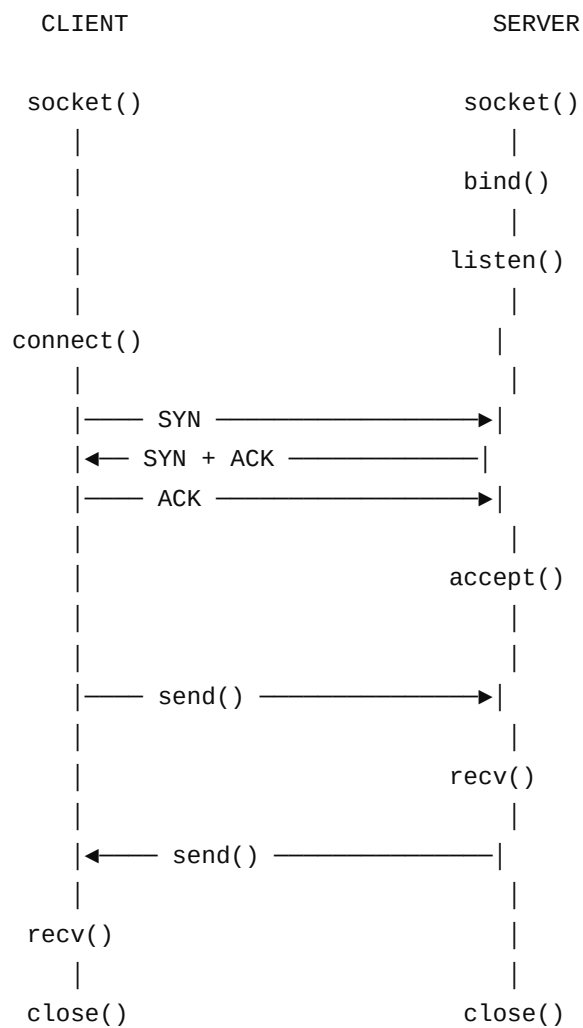
Now put everything together:

```
socket()
|
▼
bind()
|
▼
listen()
|
▼
accept()
|
▼
recv()/send()
|
▼
close()
```

32. Complete TCP Client

```
socket()
|
▼
connect()
|
▼
send()/recv()
|
▼
close()
```

33. Visualized Client → Server



This diagram is **extremely important for interviews**.

34. Full API → Network Mapping

Application API	Main purpose	Networking effect
<code>socket()</code>	Create socket	Creates kernel socket endpoint
<code>bind()</code>	Set local address/port	Associates local endpoint
<code>listen()</code>	Passive TCP listening	Prepares server for connections
<code>accept()</code>	Accept incoming connection	Returns connected socket
<code>connect()</code>	Connect to remote endpoint	TCP connection establishment
<code>send()</code>	Send application data	Data enters transport stack
<code>recv()</code>	Receive data	Gets data from receive buffer
<code>close()</code>	Close FD	Releases socket; TCP may terminate

35. The Most Important Mapping

```

socket()
  ↓
Create endpoint

bind()
  ↓
Local IP + Port

listen()
  ↓
Wait for connections

accept()
  ↓
Create connected socket

connect()
  ↓
TCP connection establishment

send()
  ↓
Application → TCP

recv()

```

↓
TCP → Application

close()

↓
Close endpoint

36. Hands-On TCP Server in C

Create:

nano server.c

Use:

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

int main()
{
    int server_fd;
    int client_fd;

    struct sockaddr_in server_addr;
    char buffer[1024];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    if (server_fd < 0) {
        perror("socket");
        return 1;
    }

    memset(&server_addr, 0, sizeof(server_addr));

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(5000);
    server_addr.sin_addr.s_addr = INADDR_ANY;

    if (bind(server_fd,
            (struct sockaddr *)&server_addr,
            sizeof(server_addr)) < 0) {
        perror("bind");
        close(server_fd);
        return 1;
    }
}
```



```
}

if (listen(server_fd, 5) < 0) {
    perror("listen");
    close(server_fd);
    return 1;
}

printf("Server waiting on port 5000...\n");

client_fd = accept(server_fd, NULL, NULL);

if (client_fd < 0) {
    perror("accept");
    close(server_fd);
    return 1;
}

printf("Client connected\n");

int n = recv(client_fd, buffer, sizeof(buffer) - 1, 0);

if (n > 0) {
    buffer[n] = '\0';

    printf("Received: %s\n", buffer);

    send(client_fd, buffer, n, 0);
}

close(client_fd);
close(server_fd);

return 0;
}
```

37. Compile

```
gcc server.c -o server
```

Run:

```
./server
```

You should see:

```
Server waiting on port 5000...
```

38. Test With nc

In another terminal:

```
nc 127.0.0.1 5000
```

Then type:

```
Hello
```

Server receives:

```
Received: Hello
```

and sends it back.

39. Observe the Connection

While server is running:

```
ss -ltn
```

You should see something conceptually like:

```
LISTEN
0.0.0.0:5000
```

This tells you:

```
TCP
↓
Port 5000
```



40. Observe the Connection After nc

While nc is connected:

```
ss -tn
```

You may see:

```
ESTAB
127.0.0.1:5000
127.0.0.1:<client-port>
```

Now connect your theory:

```
socket()
  ↓
bind(5000)
  ↓
listen()
  ↓
accept()
  ↓
TCP ESTABLISHED
```

41. Loopback Is Still Networking

Notice we're using:

```
127.0.0.1
```

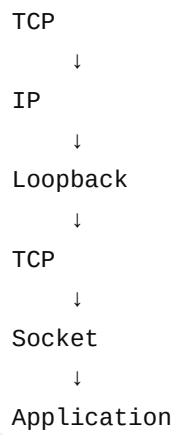
This is the **loopback address**.

The communication stays inside the host rather than going through the physical Ethernet/Wi-Fi interface.

Conceptually:

```
Application
  ↓
Socket
  ↓
```



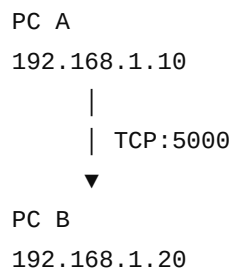


```
graph TD; TCP1[TCP] --> IP[IP]; IP --> Loopback[Loopback]; Loopback --> TCP2[TCP]; TCP2 --> Socket[Socket]; Socket --> Application[Application]
```

This is useful for your first hands-on test.

42. Test With Two Machines Later

Once localhost works:



```
graph TD; A["PC A  
192.168.1.10"] -- "TCP:5000" --> B["PC B  
192.168.1.20"]
```

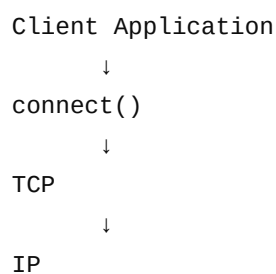
Server:

```
bind(... port 5000 ...)  
listen()  
accept()
```

Client:

```
connect(... 192.168.1.20:5000 ...)
```

Then the complete network path becomes:



```
graph TD; Client[Client Application] --> connect[connect()]; connect --> TCP[TCP]; TCP --> IP[IP]
```

↓
ARP
↓
Ethernet
↓
Switch
↓
Server NIC
↓
IP
↓
TCP
↓
accept()
↓
Server Application

★ 43. Interview Trap — socket() vs connect()

Wrong:

socket() establishes the TCP connection.

Correct:

socket() creates a socket endpoint. connect() on a TCP socket initiates a connection to the remote endpoint.

★ 44. Interview Trap — bind() vs connect()

bind()

Local endpoint

Example:

192.168.1.10:5000

connect()

Remote endpoint

Example:

192.168.1.20:5000

Visual:



A client can also explicitly `bind()` to a local address/port, but normally the OS automatically selects an ephemeral source port if the application doesn't explicitly bind one.

★ 45. Interview Trap — `listen()` vs `accept()`

`listen()` :

Makes a socket passive and ready to receive connection requests.

`accept()` :

Retrieves a pending connection and returns a new connected socket.

```
listen()
  ↓
Waiting

accept()
  ↓
Connected client socket
```

★ 46. Interview Trap — `send()` vs TCP Segment

Wrong:

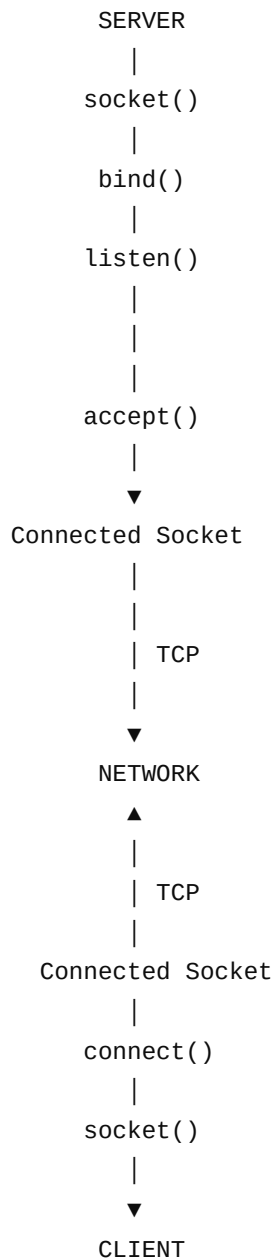
Every `send()` creates one TCP segment.

Correct:

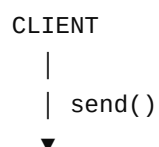
`send()` writes application data to the socket; TCP manages the byte stream and segmentation/transmission according to the protocol and current network conditions.

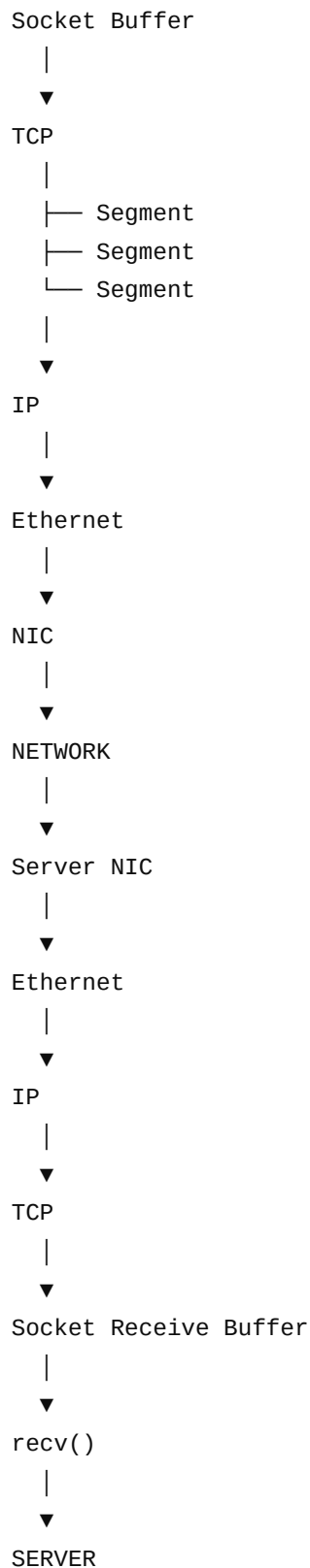
Day 13 Master Diagram

Memorize this:



And for data:





Day 13 Interview Questions

Try answering these without looking at the answers:

1. What is a socket?
2. What is the difference between socket, IP address and port?
3. What does `socket()` do?

4. What does `bind()` do?
 5. Why does a TCP server need `bind()` ?
 6. What does `listen()` do?
 7. What does `accept()` do?
 8. Why does `accept()` return a new file descriptor?
 9. What does `connect()` do?
 10. Does `socket()` establish a TCP connection?
 11. Does `bind()` establish a TCP connection?
 12. Does `connect()` directly call Layer 4?
 13. What happens internally after `connect()` ?
 14. What does `send()` do?
 15. What does `recv()` do?
 16. Does one `send()` equal one `recv()` ?
 17. Why doesn't TCP preserve message boundaries?
 18. What is `INADDR_ANY` ?
 19. Why do we use `htons()` ?
 20. Difference between listening socket and connected socket?
 21. Explain the complete TCP server API sequence.
 22. Explain the complete TCP client API sequence.
 23. Map `connect()` to SYN/SYN-ACK/ACK.
 24. Where does TCP segmentation happen?
 25. Where does IP routing happen?
-

★ Day 13 — One-Minute Interview Answer

If the interviewer says:

"Explain TCP socket programming in Linux."

Answer:

"On the server side, I create a TCP socket using `socket(AF_INET, SOCK_STREAM, 0)` . I use `bind()` to associate it with a local IP and port, then `listen()` to make it a passive listening socket. When a client connects, `accept()` returns a new connected socket. On the client side, I create a socket and use `connect()` to initiate the TCP connection. After the connection is established, both sides

use `send()` and `recv()` for data transfer, and finally `close()` to release the socket. For TCP, `connect()` results in the TCP three-way handshake, while `send()` passes application data into the TCP/IP stack."

That is a strong **Day 13 interview answer**.

Next session: Day 14 — UDP Socket Programming + TCP vs UDP in depth, including `sendto()`, `recvfrom()`, connectionless communication, broadcast/multicast basics, and when an interviewer should choose TCP vs UDP.

Try answering these without looking at the answers: 1. What is a socket? 2. What is the difference between socket, IP address and port? 3. What does `socket()` do? 4. What does `bind()` do? 5. Why does a TCP server need `bind()`? 6. What does `listen()` do? 7. What does `accept()` do?...

[ans](#)

Day 13 — Interview Questions & Answers

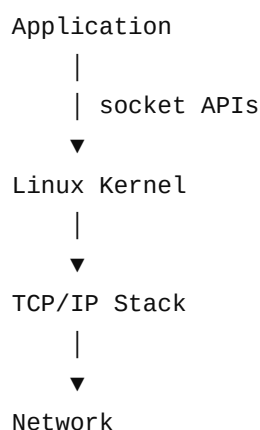
Here are the answers in **simple interview language**, followed by the deeper explanation where important.

1. What is a socket?

Answer:

A socket is an operating-system communication endpoint that an application uses to communicate with another process or machine over a network.

Example:



A socket is represented to the application by a **file descriptor**.

2. Difference between socket, IP address and port?

Term	Meaning
IP address	Identifies a host/interface on a network
Port	Identifies a service/application endpoint on a host
Socket	OS-managed communication endpoint used by an application

Example:

192.168.1.10:5000

```
graph LR
    A[192.168.1.10:5000] --> B[192.168.1.10]
    A --> C[:5000]
    B --- D[IP]
    C --- E[Port]
```

The application uses a socket to communicate through that endpoint.

Interview answer:

IP identifies the host, port identifies the service endpoint, and a socket is the OS interface through which the application communicates.

3. What does `socket()` do?

```
socket(AF_INET, SOCK_STREAM, 0);
```

It creates a socket and returns a **file descriptor**.

```
graph TD
    A[Application] --> B[socket()]
    B --> C[Kernel]
    C --> D[Socket object]
    D --> E[|]
```

It does **not** establish a TCP connection.

Important parameters

```
socket(domain, type, protocol);
```

- AF_INET → IPv4
- SOCK_STREAM → TCP-style stream socket
- 0 → select the default protocol for the specified domain/type

4. What does bind() do?

bind() associates a socket with a **local address and port**.

Example:

```
bind(sockfd, ...);
```

Conceptually:

```
Socket
  ↓
Local IP + Port
  ↓
192.168.1.10:5000
```

It tells the kernel:

"Use this local address/port for this socket."

5. Why does a TCP server need bind() ?

Because the server normally needs a **known local port** where clients can connect.

Example:

```
Server
192.168.1.10:5000
```

↑
known port

Client can then connect to:

192.168.1.10:5000

Without explicitly binding, the server doesn't normally establish the desired well-known listening endpoint.

Interview answer:

A TCP server uses `bind()` to associate its listening socket with a known local IP address and port so clients know where to connect.

6. What does `listen()` do?

`listen()` puts a TCP socket into **passive listening mode**.

```
listen(sockfd, 5);
```

Flow:

```
socket()
  ↓
bind()
  ↓
listen()
  ↓
Wait for clients
```

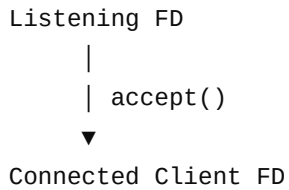
It does **not** create a connection with a specific client.

7. What does `accept()` do?

`accept()` accepts a pending incoming connection and returns a **new connected socket file descriptor**.

```
client_fd = accept(server_fd, ...);
```

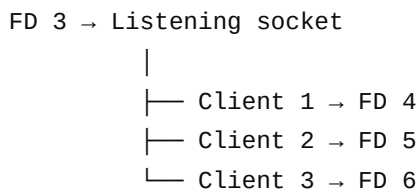
Visual:



8. Why does `accept()` return a new file descriptor?

Because the original socket is the **listening socket** and must remain available to accept other clients.

Example:



This allows one server to handle multiple clients.

Very important interview point:

`accept()` does not convert the listening socket into the client connection. It returns a new socket for that connection.

9. What does `connect()` do?

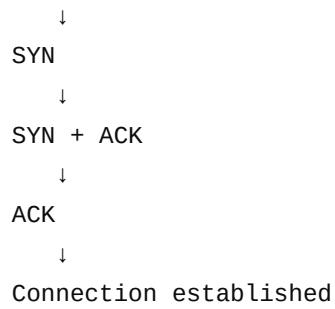
`connect()` requests a connection to a remote endpoint.

Example:

```
connect(sockfd, ...);
```

For TCP:

```
graph TD; A[connect()] --> B[TCP]
```

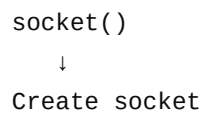


```
graph TD; A[↓] --> B[SYN]; B --> C[↓]; C --> D[SYN + ACK]; D --> E[↓]; E --> F[ACK]; F --> G[↓]; G --> H[Connection established];
```

So `connect()` is associated with **TCP connection establishment**.

10. Does `socket()` establish a TCP connection?

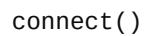
No.



```
graph TD; A[socket()] --> B[Create socket];
```

It doesn't contact the remote server.

The TCP connection is initiated later using:



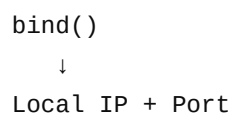
```
graph TD; A[connect()];
```

on the client side.

11. Does `bind()` establish a TCP connection?

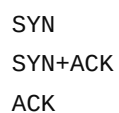
No.

`bind()` only associates the socket with a **local address/port**.



```
graph TD; A[bind()] --> B[Local IP + Port];
```

It does not perform:



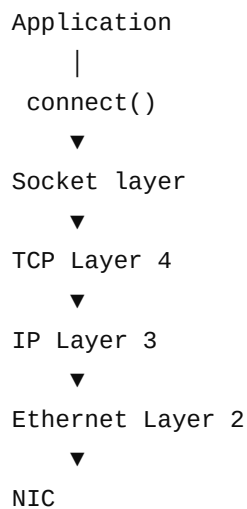
```
graph TD; A[SYN] --> B[SYN+ACK] --> C[ACK];
```

12. Does `connect()` directly call Layer 4?

The best interview answer is:

`connect()` is a socket API exposed by the kernel. For a TCP socket, the kernel's networking stack processes the request using TCP at the transport layer and initiates the TCP connection.

Visualize:



So don't say:

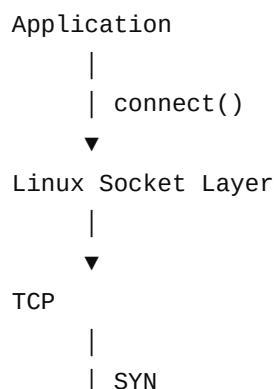
" `connect()` directly calls Layer 4."

Say:

" `connect()` is a socket API that causes the kernel's TCP stack to initiate the connection."

13. What happens internally after `connect()` ?

For a TCP connection, simplified flow:





Server responds:

SYN + ACK

Client responds:

ACK

Then:

TCP connection established

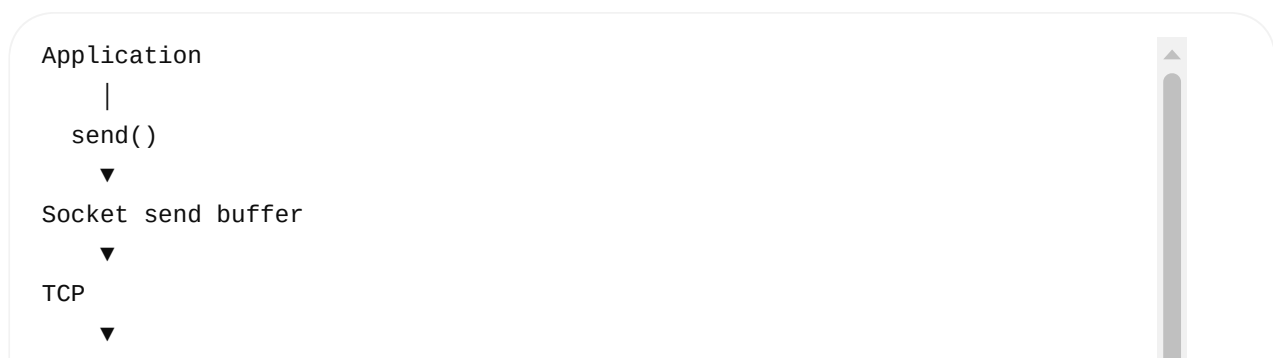
14. What does `send()` do?

`send()` passes application data to the socket.

Example:

```
send(fd, buffer, length, 0);
```

Conceptually:



```
graph TD; A[TCP segments] --> B[IP packets]; B --> C[Ethernet frames]; C --> D[NIC];
```

TCP segments
▼
IP packets
▼
Ethernet frames
▼
NIC

Important:

`send()` does not itself construct an Ethernet frame in user space. The kernel networking stack handles the lower-level processing.

15. What does `recv()` do?

`recv()` reads available data from the socket's receive side.

```
recv(fd, buffer, size, 0);
```

Conceptually:

```
graph TD; A[NIC] --> B[Ethernet]; B --> C[IP]; C --> D[TCP]; D --> E[Socket receive buffer]; E --> F[recv()]; F --> G[Application];
```

NIC
↓
Ethernet
↓
IP
↓
TCP
↓
Socket receive buffer
↓
`recv()`
↓
Application

16. Does one `send()` equal one `recv()` ?

No.

This is one of the most important TCP interview questions.

Suppose:

```
send(fd, "Hello", 5, 0);  
send(fd, "World", 5, 0);
```

The receiver could get:

HelloWorld

in one `recv()` .

Or:

Hello

then:

World

Or even another grouping.

TCP is a **byte stream**, not a message-oriented protocol.

17. Why doesn't TCP preserve message boundaries?

Because TCP provides an ordered, reliable **byte stream**.

It does not know that your application performed:

```
send("Hello")  
send("World")
```

It sees a sequence of bytes:

HelloWorld

Therefore the application protocol must define message boundaries if required.

Common approaches:

Length prefix

or:

Delimiter

or:

Fixed-size message

Example:

[5][Hello][5][World]

18. What is INADDR_ANY ?

Example:

```
server_addr.sin_addr.s_addr = INADDR_ANY;
```

It tells the IPv4 server socket to bind to the machine's local IPv4 addresses rather than one specific local address.

Conceptually:

Server

- |— 127.0.0.1
- |— 192.168.1.10
- |— 10.0.0.5

INADDR_ANY

↓

Listen on local IPv4 interfaces

For an interview:

INADDR_ANY allows a server to bind to all suitable local IPv4 interfaces.

19. Why do we use htons() ?

htons() means:

Host TO Network Short

Example:

```
server_addr.sin_port = htons(5000);
```

Network protocols use **network byte order**, which is big-endian.

Your CPU may use a different byte order.

So:

```
Host byte order
  ↓
  htons()
  ↓
Network byte order
```

Common functions:

API	Meaning
htons()	host → network, 16-bit
htonl()	host → network, 32-bit
ntohs()	network → host, 16-bit
ntohl()	network → host, 32-bit

20. Listening socket vs connected socket

This is very important.

Listening socket

```
socket()
  ↓
bind()
  ↓
listen()
```

Purpose:

Wait for incoming connections.

Connected socket

Created/obtained through:

`accept()`

Purpose:

Communicate with a particular client.

Visual:

Server

FD 3

|

└─ Listening socket

|

└─ `accept()` → FD 4 → Client 1

└─ `accept()` → FD 5 → Client 2

└─ `accept()` → FD 6 → Client 3

21. Explain the complete TCP server API sequence.

Interview answer:

First, the server calls `socket()` to create a TCP socket. Then it calls `bind()` to associate the socket with a local IP address and port. `listen()` puts the socket into passive listening mode. `accept()` waits for an incoming connection and returns a new connected socket. The server then uses `recv()` and `send()` on that connected socket for data communication. Finally, it calls `close()`.

Diagram:

`socket()`

↓

`bind()`

↓

`listen()`

↓

`accept()`

↓

`recv()/send()`

↓

`close()`

22. Explain the complete TCP client API sequence.

Interview answer:

The client creates a socket using `socket()`, then calls `connect()` with the server's IP address and port. For TCP, this initiates the TCP three-way handshake. After the connection is established, the client uses `send()` and `recv()` for communication and finally closes the socket.

Diagram:

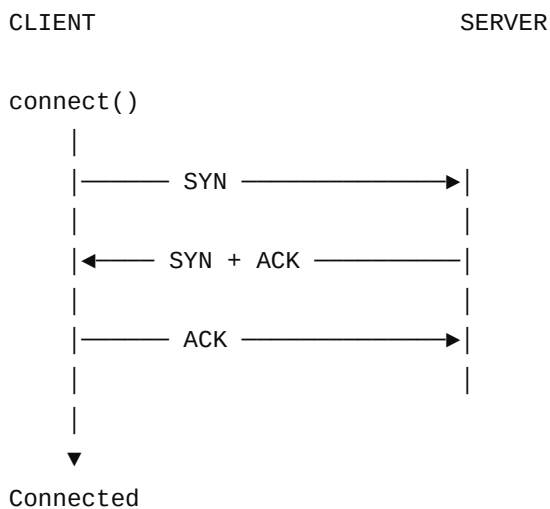
```
socket()
↓
connect()
↓
TCP handshake
↓
send()/recv()
↓
close()
```

23. Map `connect()` to SYN/SYN-ACK/ACK.

When the client executes:

```
connect()
```

the kernel's TCP stack initiates:



So:

```
connect()
  ↓
SYN
  ↓
SYN + ACK
  ↓
ACK
  ↓
TCP ESTABLISHED
```

24. Where does TCP segmentation happen?

TCP segmentation happens in the **transport layer**, implemented by the OS networking stack.

Conceptually:

```
Application data
  |
  ▼
TCP
  |
  ├── Segment 1
  ├── Segment 2
  └── Segment 3
```

Then:

```
TCP Segment
  ↓
IP Packet
  ↓
Ethernet Frame
```

One important refinement:

The exact packetization can also be affected by mechanisms such as offloads in modern NICs, so the exact point at which physical segmentation is performed may not always correspond directly to the conceptual TCP-layer diagram.

For an interview, **"TCP handles segmentation at Layer 4"** is the correct conceptual answer.

25. Where does IP routing happen?

IP routing is a **Layer 3 function**.

On Linux, the kernel checks its routing table.

For example:

```
ip route
```

Conceptually:

```
Destination IP
  ↓
Routing table
  ↓
Next hop / outgoing interface
```

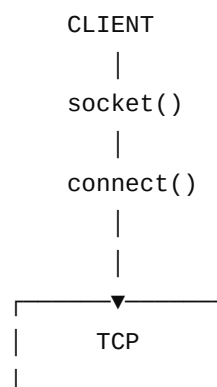
Example:

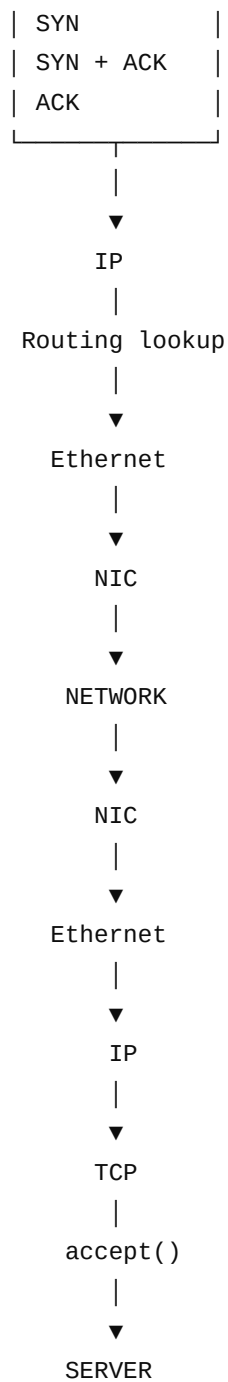
```
Application
  ↓
TCP
  ↓
IP
  ↓
Routing lookup
  ↓
eth0
  ↓
NIC
```

A router performs similar Layer 3 forwarding decisions between networks.

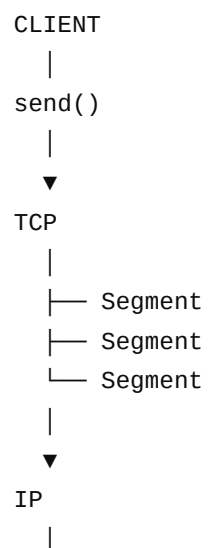
🔥 Complete Day 13 Flow

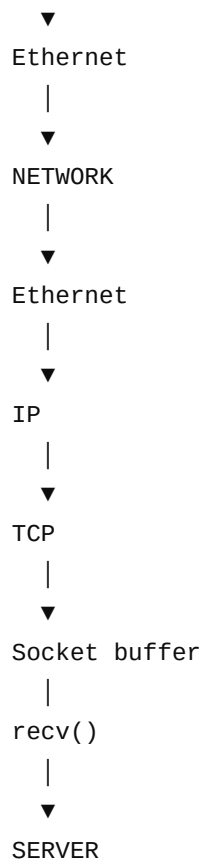
Now combine everything you've learned:





Then data:





★ The 10 things you should remember for the interview

1. `socket()` → create endpoint
2. `bind()` → local IP + port
3. `listen()` → wait for connections
4. `accept()` → new connected socket
5. `connect()` → initiate TCP connection
6. `send()` → application → socket
7. `recv()` → socket → application
8. `close()` → close socket
9. TCP → byte stream, not message boundaries
10. Routing → Layer 3 / IP forwarding decision

And the **server sequence**:

`socket → bind → listen → accept → send/recv → close`

The **client sequence**:

`socket → connect → send/recv → close`

These two sequences are the core of **Day 13**.